

Abstract Factory

Definition

Class Diagram

Example: Car Manufacturing

Implementation

Factory Method vs Abstract Factory

▼ Resources

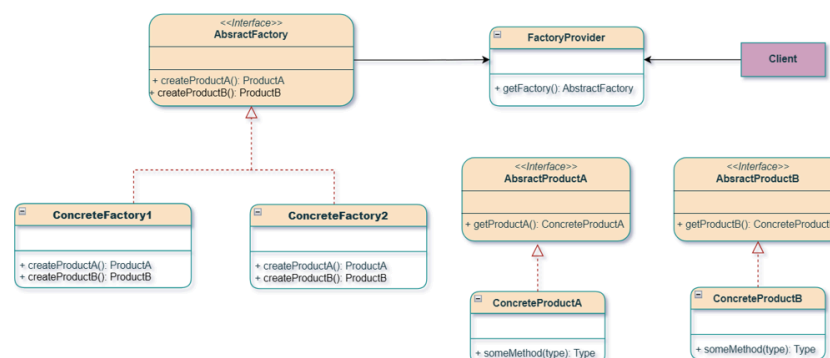
- Video → [27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)
- Video → [5. Factory Pattern Vs Abstract Factory Pattern Explanation \(Hindi\) | Low Level System Design](#)

Definition

The Abstract Factory Pattern provides an **interface** for **creating families of related objects** without specifying their **concrete classes**. It's like having multiple factories, where each factory produces a complete family of products that **work together/are related/dependent**.

As a result, an Abstract Factory is known as the Factory of Factories or a Super Factory. It is more complex than the Factory pattern, as it is responsible for creating families of multiple related product types. Hence, we see multiple factory methods in the interface. This high level of abstraction enables the client to switch between different product families at runtime without knowing the concrete implementation of product instantiation.

Class Diagram



Example: Car Manufacturing

1. Abstract Product Interfaces(CarInterior , CarExterior)

- Define interfaces for a family of related products.
- Each product type has its interface.
- Clients depend on this interface to create objects without needing to know their specific types.

2. Concrete Products (EconomyCarInterior , LuxuryCarExterior ,etc.)

- Implement the abstract product interfaces.
- Products from the same family are designed to work together.

3. **Abstract Factory (CarFactory)**

- Declares creation methods for each product type.
- Defines the contract for creating product families.

4. **Concrete Factories (EconomyCarFactory , LuxuryCarFactory)**

- Implement the abstract factory interface.
- Each factory produces products from a single specific family.

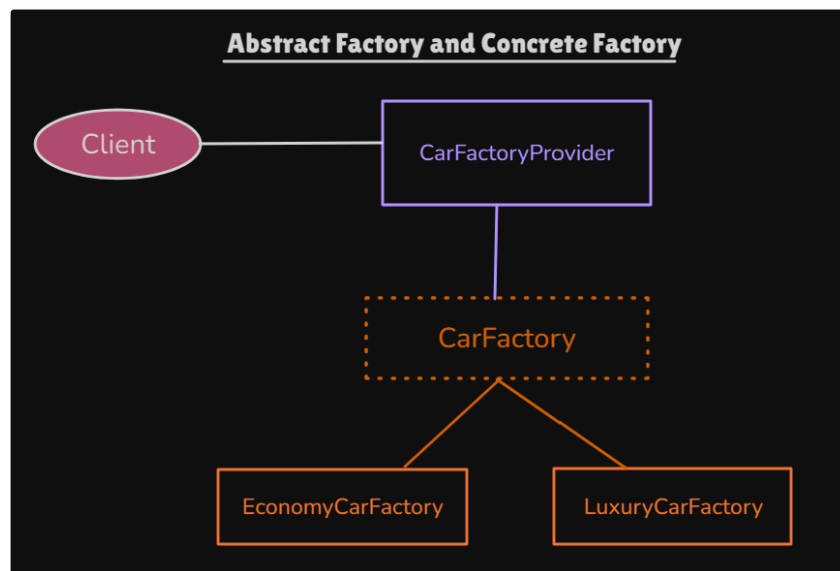
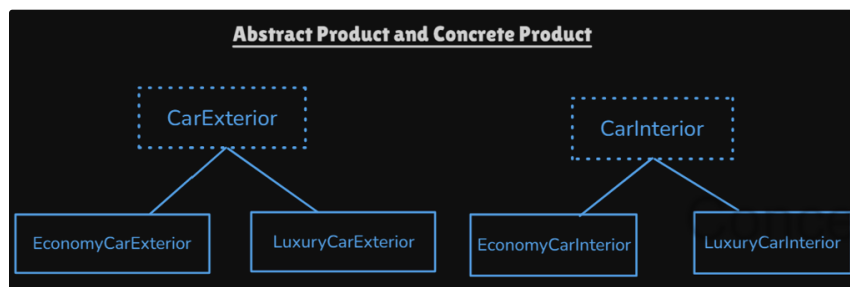
5. **Factory Provider**

- Returns a concrete factory based on client input in the request.
- This concrete factory is used to create families of concrete products as per the client's request.

6. **Client (Application)**

- Utilizes the **Abstract Factory** Interface to request concrete products.

Implementation



1. **Abstract Product Interfaces**

```

1 // Step 1: Abstract Product interfaces - Define product families
2 public interface CarExterior {
3     void addExteriorComponents();
4 }
5 public interface CarInterior {
6     void addInteriorComponents();
  
```

```
7 }
```

2. Concrete Products

```
1 // Step 2: Concrete Products for Economy Car Family
2 public class EconomyCarExterior implements CarExterior {
3     @Override
4     public void addExteriorComponents() {
5         System.out.println("Adding basic exterior components for
Economy Car.");
6     }
7 }
8 public class EconomyCarInterior implements CarInterior {
9     @Override
10    public void addInteriorComponents() {
11        System.out.println("Adding basic interior components for
Economy Car.");
12    }
13 }
14 // Step 3: Concrete Products for Luxury Car Family
15 public class LuxuryCarExterior implements CarExterior {
16     @Override
17     public void addExteriorComponents() {
18         System.out.println("Adding luxurious exterior components for
Luxury Car.");
19     }
20 }
21 public class LuxuryCarInterior implements CarInterior {
22     @Override
23     public void addInteriorComponents() {
24         System.out.println("Adding luxurious interior components for
Luxury Car.");
25     }
26 }
```

3. Abstract Factory

Concept && Coding

```
1 // Step 4: Abstract Factory interface
2 public interface CarFactory {
3
4     // Factory methods
5     CarInterior createInterior();
6
7     CarExterior createExterior();
8
9     // Template method that uses all factory methods
10    default void produceCompleteVehicle() {
11
12        System.out.println("Starting complete vehicle production...");
13
14        // Create all components
15        CarInterior interior = createInterior();
16        CarExterior exterior = createExterior();
17
18        // Assemble the vehicle
19        interior.addInteriorComponents();
20        exterior.addExteriorComponents();
21
22        System.out.println("Vehicle production completed!");
23    }
24 }
```

4. Concrete Factories

```
1 // Step 5: Concrete Factories
2 public class EconomyCarFactory implements CarFactory {
3     private String brand;
4
5     public EconomyCarFactory(String brand) {
6         this.brand = brand;
```

```

7     }
8     @Override
9     public CarInterior createInterior() {
10         return new EconomyCarInterior();
11     }
12
13     @Override
14     public CarExterior createExterior() {
15         return new EconomyCarExterior();
16     }
17 }
18 // Step 5: Concrete Factories
19 public class LuxuryCarFactory implements CarFactory {
20     private final String brand;
21
22     public LuxuryCarFactory(String brand) {
23         this.brand = brand;
24     }
25
26     @Override
27     public CarInterior createInterior() {
28         return new LuxuryCarInterior();
29     }
30
31     @Override
32     public CarExterior createExterior() {
33         return new LuxuryCarExterior();
34     }
35 }

```

5. Factory Provider

```

1 // Step 6: Factory Provider
2 public class CarFactoryProvider {
3     public CarFactory getFactory(CarType type, String brand) {
4         switch (type) {
5             case ECONOMY:
6                 return new EconomyCarFactory(brand);
7             case PREMIUM:
8             case LUXURY:
9                 return new LuxuryCarFactory(brand);
10            default:
11                throw new IllegalArgumentException("Unknown car type:
12 " + type);
13        }
14    }
15 }

```

6. Client

```

1 // Step 7: Client Application
2 public class AbstractFactoryDemo {
3     public static void main(String[] args) {
4         System.out.println("====Abstract Factory Design
5 Pattern====");
6         // Get Factory Provider
7         CarFactoryProvider carFactoryProvider = new
8 CarFactoryProvider();
9
10        // Get Economy Car Factory
11        CarFactory economyCar =
12 carFactoryProvider.getFactory(CarType.ECONOMY, "Honda");
13        economyCar.produceCompleteVehicle();
14
15        // Get Luxury Car Factory
16        CarFactory luxuryCar =
17 carFactoryProvider.getFactory(CarType.LUXURY, "Mercedes");
18        luxuryCar.produceCompleteVehicle();
19
20        // Get Premium Car Factory

```

```
17         CarFactory premiumCar =
18             carFactoryProvider.getFactory(CarType.PREMIUM, "Rolls Royce");
19             premiumCar.produceCompleteVehicle();
20     }
```

Output

```
====Abstract Factory Design Pattern====
Starting complete vehicle production...
Adding basic interior components for Economy Car.
Adding basic exterior components for Economy Car.
Vehicle production completed!
Starting complete vehicle production...
Adding luxurious interior components for Luxury Car.
Adding luxurious exterior components for Luxury Car.
Vehicle production completed!
Starting complete vehicle production...
Adding luxurious interior components for Luxury Car.
Adding luxurious exterior components for Luxury Car.
Vehicle production completed!

Process finished with exit code 0
```

The Abstract Factory Pattern is highly effective when you need to ensure that related products work well together, making it especially suitable for complex systems with various product variants. This highlights the pattern's strong extensibility and its compliance with the Open/Closed Principle.

Factory Method vs Abstract Factory

Refer → [5. Factory Pattern Vs Abstract Factory Pattern Explanation \(Hindi\) | Low Level System Design](#)

Summary:

- One product, many variants → Use **Factory Method**
- Many products, grouped by family (theme, platform, brand, etc.) → Use **Abstract Factory**

Builder

[Definition](#)

[The Problem](#)

[Class Diagram](#)

[Example: Student Registration](#)

[Implementation](#)

[Builder Pattern vs Decorator Pattern](#)

Resources

- Video → [27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)
- Video → [23. Builder Design Pattern with Examples, LLD | Low Level Design Interview Question | System Design](#)

Definition

The *Builder Pattern* is a creational design pattern that constructs *complex objects* *step by step*.

It's particularly useful when you need to create objects with many optional parameters or when the construction process should allow different representations of the same object.

Concept && Coding

The Problem

```
1 // Step 1: Product class - The complex Student object being built
2 public class Student {
3
4     // mandatory fields
5     int rollNumber;
6     int age;
7     String name;
8     String branch;
9     // optional fields
10    String fatherName;
11    String motherName;
12    List<String> subjects;
13    String mobileNo;
14    String emailId;
15
16    // Without Builder - Constructor overloading
17    // Telescoping Constructor Problem - adding mandatory parameters
18    public Student(int rollNumber, int age, String name, String
branch) {
19        this.rollNumber = rollNumber;
20        this.age = age;
21        this.name = name;
22        this.branch = branch;
23    }
24
25    // Telescoping Constructor Problem - adding new parameter -
fatherName(optional)
26    public Student(int rollNumber, int age, String name, String
branch, String fatherName) {
27        this.rollNumber = rollNumber;
28        this.age = age;
```

```

29         this.name = name;
30         this.branch = branch;
31         this.fatherName = fatherName;
32     }
33
34     // Telescoping Constructor Problem - adding new parameter -
    motherName(optional)
35     public Student(int rollNumber, int age, String name, String
    branch, String fatherName, String motherName) {
36         this.rollNumber = rollNumber;
37         this.age = age;
38         this.name = name;
39         this.branch = branch;
40         this.fatherName = fatherName;
41         this.motherName = motherName;
42     }
43
44     // Telescoping Constructor Problem - adding new parameter -
    emailId(optional)
45     // Compilation error - Constructor signature is same as another
    constructor(below)
46     public Student(int rollNumber, int age, String name, String
    branch, String fatherName, String motherName, String emailId) {
47         this.rollNumber = rollNumber;
48         this.age = age;
49         this.name = name;
50         this.branch = branch;
51         this.fatherName = fatherName;
52         this.motherName = motherName;
53         this.emailId = emailId;
54     }
55
56     // Telescoping Constructor Problem - adding new parameter -
    emailId(optional)
57     // Compilation error - Constructor signature is same as another
    constructor(above)
58     /*public Student(int rollNumber, int age, String name, String
    branch, String fatherName, String motherName, String mobileNo) {
59         this.rollNumber = rollNumber;
60         this.age = age;
61         this.name = name;
62         this.branch = branch;
63         this.fatherName = fatherName;
64         this.motherName = motherName;
65         this.mobileNo = mobileNo;
66     }*/
67
68     // Telescoping Constructor Problem - All arguments constructor
69     // Big Constructor with long parameter list
70     public Student(int rollNumber, int age, String name, String
    branch, String fatherName, String motherName, List<String> subjects,
    String mobileNo, String emailId) {
71         this.rollNumber = rollNumber;
72         this.age = age;
73         this.name = name;
74         this.branch = branch;
75         this.fatherName = fatherName;
76         this.motherName = motherName;
77         this.subjects = subjects;
78         this.mobileNo = mobileNo;
79         this.emailId = emailId;
80     }
81
82     public void printDetails() {
83         System.out.println("== Student Details ==");
84         System.out.print(this + ": ");
85         System.out.println("Id: " + rollNumber +
86             ", Name: " + name +
87             ", Age: " + age +
88             ", Branch: " + branch +
89             ", Roll No: " + rollNumber +

```

Concept & Coding

```

90         ", Father Name: " + fatherName +
91         ", Mother Name: " + motherName +
92         ", Subjects: " + subjects +
93         ", Mobile No: " + mobileNo +
94         ", Email Id: " + emailId);
95     }
96 }

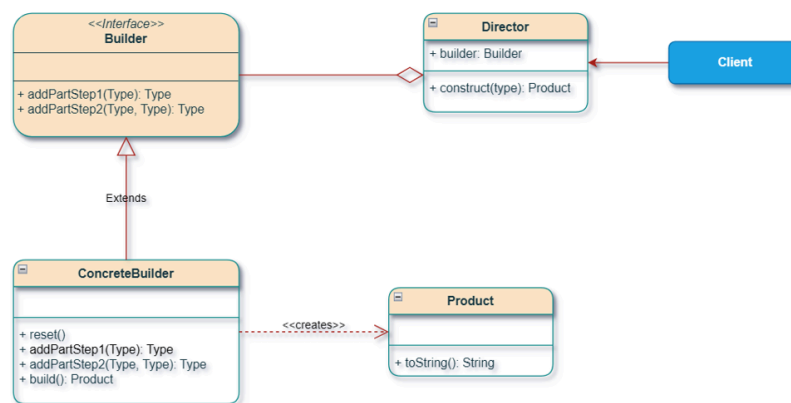
```

Here's a list of problems and issues that typically arise when we create similar and multiple complex objects, the traditional way.

- When a class has many optional parameters, you end up with a large number of constructors. This is called Telescoping Constructors (Constructor Overload Explosion).
- It is difficult to maintain, read, and prone to errors when parameters are of the same type, particularly when constructors have identical signatures.
- Reduced readability and maintainability, as we can't tell what each parameter means unless we look at the constructor definition. Chances of wrong assignment of parameter values due to the same types.
- Clients should pass `null` for any optional parameters that they do not choose to set, which increases the risk of bugs.
- If you want to make a class immutable (i.e., its fields are final), you cannot use setters.
- Creating test objects becomes increasingly complex and verbose without builders, particularly when many fields are optional.
- Violates the Single Responsibility Principle (SRP) because the class manages both data storage and construction logic.

Concept & Coding

Class Diagram



Example: Student Registration

1. Product (Student)

- The `product` is a complex object that is being constructed.
- Contains many optional and required components.
- Has package-private setters to ensure only builders can modify it.

2. Abstract Builder Interface (StudentBuilder)

- Defines all possible construction steps.
 - Provides a fluent interface (for method chaining).
 - Includes a `build()` method to return the final product(`Student`).
3. **Concrete Builders (`EngineeringStudentBuilder` and `MBASStudentBuilder`)**
- Implement the builder interface.
 - Each builder creates a specific type of product.
 - Contain the logic for how each component should be built for that specific product type.
4. **Director (`StudentRegistrationDirector`) - Optional**
- Defines common construction sequences.
 - Encapsulates the order of construction steps.
 - Provides convenience methods for standard configurations.

Implementation

```

1 // Step 1: Product class - Complex Student object
2 public class Student {
3
4     // mandatory fields
5     int rollNumber;
6     int age;
7     String name;
8     String branch;
9     // optional fields
10    String fatherName;
11    String motherName;
12    List<String> subjects;
13    String mobileNo;
14    String emailId;
15
16    // Constructor - package private, only the builder can create
17    Student(StudentBuilder builder) {
18        this.rollNumber = builder.rollNumber;
19        this.age = builder.age;
20        this.name = builder.name;
21        this.branch = builder.branch;
22        this.fatherName = builder.fatherName;
23        this.motherName = builder.motherName;
24        this.subjects = builder.subjects;
25        this.mobileNo = builder.mobileNo;
26        this.emailId = builder.emailId;
27    }
28
29    public String toString() {
30        return " roll number: " + this.rollNumber +
31            " age: " + this.age +
32            " name: " + this.name +
33            " branch: " + this.branch +
34            " father name: " + this.fatherName +
35            " mother name: " + this.motherName +
36            " subjects: " + subjects.get(0) + "," +
37            subjects.get(1) + "," + subjects.get(2) +
38            " mobile no: " + this.mobileNo +
39            " email id: " + this.emailId;
40    }
41 }

```

```

1 // Step 2: Abstract Builder interface
2 public abstract class StudentBuilder {
3

```

Concept & Coding

```

4      // mandatory fields
5      int rollNumber;
6      int age;
7      String name;
8      String branch;
9      // optional fields
10     String fatherName;
11     String motherName;
12     List<String> subjects;
13     String mobileNo;
14     String emailId;
15
16     public StudentBuilder setRollNumber(int rollNumber) {
17         this.rollNumber = rollNumber;
18         return this;
19     }
20
21     public StudentBuilder setAge(int age) {
22         this.age = age;
23         return this;
24     }
25
26     public StudentBuilder setName(String name) {
27         this.name = name;
28         return this;
29     }
30
31     public StudentBuilder setBranch(String branch) {
32         this.branch = branch;
33         return this;
34     }
35
36     public StudentBuilder setFatherName(String fatherName) {
37         this.fatherName = fatherName;
38         return this;
39     }
40
41     public StudentBuilder setMotherName(String motherName) {
42         this.motherName = motherName;
43         return this;
44     }
45
46     public StudentBuilder setMobileNo(String mobileNo) {
47         this.mobileNo = mobileNo;
48         return this;
49     }
50
51     public StudentBuilder setEmailId(String emailId) {
52         this.emailId = emailId;
53         return this;
54     }
55
56     abstract public StudentBuilder setSubjects();
57
58     // Build method
59     public Student build() {
60         return new Student(this);
61     }
62
63 }

```

```

1 // Step 3: Concrete Builder for Engineering Students
2 public class EngineeringStudentBuilder extends StudentBuilder {
3
4     // Engineering-specific methods
5     @Override
6     public StudentBuilder setSubjects() {
7         List<String> enggSubjectsList = new ArrayList<>();
8         enggSubjectsList.add("Operating Systems");
9         enggSubjectsList.add("Computer Architecture");
10        enggSubjectsList.add("Data Structures");

```

```

11     enggSubjectsList.add("DBMS");
12     this.subjects = enggSubjectsList;
13     return this;
14 }
15 }
16
17 // Step 4: Concrete Builder for MBA Students
18 public class MBASStudentBuilder extends StudentBuilder {
19
20     // MBA-specific methods
21     @Override
22     public StudentBuilder setSubjects() {
23         List<String> mbaSubjectsList = new ArrayList<>();
24         mbaSubjectsList.add("Micro Economics");
25         mbaSubjectsList.add("Business Studies");
26         mbaSubjectsList.add("Operations Management");
27         mbaSubjectsList.add("Financial Management");
28         this.subjects = mbaSubjectsList;
29         return this;
30     }
31 }

```

```

1 // Step 5: Director class for common student registration processes
2 public class StudentRegistrationDirector {
3
4     StudentBuilder studentBuilder;
5
6     StudentRegistrationDirector(StudentBuilder studentBuilder) {
7         this.studentBuilder = studentBuilder;
8     }
9
10    public Student createStudent() {
11        if (studentBuilder instanceof EngineeringStudentBuilder) {
12            return createEngineeringStudent();
13        } else if (studentBuilder instanceof MBASStudentBuilder) {
14            return createMBASStudent();
15        }
16        return null;
17    }
18
19    private Student createEngineeringStudent() {
20        return studentBuilder.setRollNumber(1) //
21        EngineeringStudentBuilder
22            .setAge(22)
23            .setName("John")
24            .setFatherName("Paul")
25            .setMotherName("Jane")
26            .setBranch("Computer Science and Engineering")
27            .setSubjects() // Engineering-specific method
28            .build();
29    }
30
31    private Student createMBASStudent() {
32        return studentBuilder.setRollNumber(2) // MBASStudentBuilder
33            .setAge(24)
34            .setName("Sarah")
35            .setFatherName("Gabriel")
36            .setMotherName("Taylor")
37            .setBranch("Business Administration")
38            .setSubjects() // MBA-specific method
39            .setMobileNo("9876543210")
40            .setEmailId("sarahgabriel@iitb.com")
41            .build();
42    }
43 }

```

```

1 // Step 6: Client demonstration
2 public class Client {
3
4     public static void main(String[] args) {
5         System.out.println("==== Builder Pattern =====");
6     }
7 }

```

```

6      // Create director objects
7      StudentRegistrationDirector enggStudentDirector = new
StudentRegistrationDirector(new EngineeringStudentBuilder());
8      StudentRegistrationDirector mbaStudentDirector = new
StudentRegistrationDirector(new MBASStudentBuilder());
9
10     // Create students using different builders
11     Student engineerStudent = enggStudentDirector.createStudent();
12     Student mbaStudent = mbaStudentDirector.createStudent();
13
14     // Print student details
15     System.out.println("==> Student details:" +
engineerStudent.toString());
16     System.out.println("==> Student details:" +
mbaStudent.toString());
17
18     }
19 }

```

Builder Pattern vs Decorator Pattern

Refer → [📺 23. Builder Design Pattern with Examples, LLD | Low Level Design Interview Question | System Design](#)

Conclusion:

- If you're building complex objects: **Use the Builder Pattern**
- If you're extending behavior dynamically(layering enhancements): **Use the Decorator Pattern**

Concept && Coding

Factory

Definition

Class Diagram

Example: Shape

Implementation

Simple Factory

Factory Method

Factory Method vs Simple Factory

▼ Resources

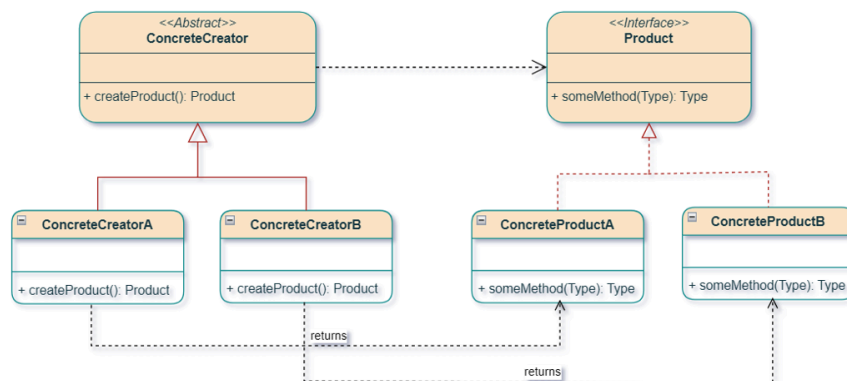
- Video → [27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)
- Video → [5. Factory Pattern Vs Abstract Factory Pattern Explanation \(Hindi\) | Low Level System Design](#)

Definition

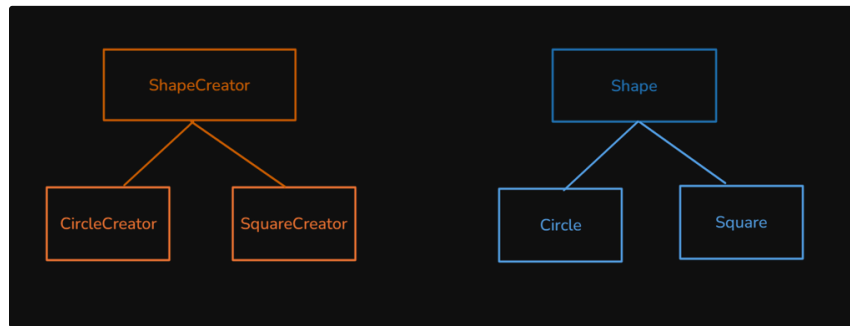
The Factory Method Pattern is used when we want to encapsulate object creation, instantiation, and all related business logic in one place.

This approach offers an interface for creating objects without needing to specify their exact classes. It delegates the responsibility of object creation to subclasses, which implement specific functionalities. Instead of using the "new" keyword directly to create objects, we utilize a factory method that returns objects adhering to a common interface. The actual class that gets instantiated is determined at runtime. This pattern encourages loose coupling and enhances extensibility.

Class Diagram



Example: Shape



1. Product (Shape)

- Defines the interface for objects that the factory method creates
- All concrete shapes implement this interface

2. Concrete Products (Circle, Square, Rectangle, Triangle)

- Specific implementations of the Shape interface
- Each has its own drawing logic and area calculation

3. Creator (ShapeFactory)

- Abstract class that declares the factory method
- May provide default implementation or template methods

4. Concrete Creators (CircleCreator, RectangleCreator, etc.)

- Override the factory method to return specific product instances
- Each factory knows how to create one type of shape

Concept && Coding

Implementation

```
1 // Step 1: Define the Product interface
2 public interface Shape {
3     void computeArea();
4     void draw();
5 }
6
7 // Step 2: Concrete Product classes
8 public class Circle implements Shape {
9
10     @Override
11     public void computeArea() {
12         System.out.println("Inside Circle::computeArea() method.");
13     }
14
15     @Override
16     public void draw() {
17         System.out.println("Inside Circle::draw() method.");
18     }
19 }
20 public class Rectangle implements Shape {
21
22     @Override
23     public void computeArea() {
24         System.out.println("Inside Rectangle::computeArea() method.");
25     }
26
27     @Override
28     public void draw() {
```

```

29         System.out.println("Inside Rectangle::draw() method.");
30     }
31 }
32 public class Square implements Shape {
33
34     @Override
35     public void computeArea() {
36         System.out.println("Inside Square::computeArea() method.");
37     }
38
39     @Override
40     public void draw() {
41         System.out.println("Inside Square::draw() method.");
42     }
43 }

```

Simple Factory

```

1 public enum ShapeType {
2     CIRCLE, RECTANGLE, SQUARE
3 }
4 public class ShapeFactory {
5     public static Shape createShapeInstance(ShapeType shapeType) {
6         if (shapeType == null) {
7             return null;
8         }
9         return switch (shapeType) {
10             case CIRCLE -> new Circle();
11             case RECTANGLE -> new Rectangle();
12             case SQUARE -> new Square();
13             default -> throw new IllegalStateException("ShapeType
doesn't exist!");
14         };
15     }
16 }

```

Concept & Coding

```

1 // Step 3: Simple Factory Demo (Bloated Design)
2 public class SimpleFactoryDemo {
3     public static void main(String[] args) {
4         System.out.println("===== Simple Factory Design Pattern
=====");
5
6         // set the type you want
7         ShapeType shapeType = ShapeType.SQUARE;
8         // get the shape
9         Shape shape = ShapeFactory.createShapeInstance(shapeType);
10        shape.draw();
11        shape.computeArea();
12    }
13 }

```

Factory Method

```

1 // Step 3: Abstract Creator class
2 public abstract class ShapeFactory {
3     // Factory method - to be implemented by subclasses
4     public abstract Shape createShape();
5 }
6
7 // Step 4: Concrete Creator classes
8 public class CircleCreator extends ShapeFactory {
9
10    @Override
11    public Shape getShapeInstance() {
12        return new Circle();
13    }
14 }
15 public class RectangleCreator extends ShapeFactory {
16

```

```

17     @Override
18     public Shape getShapeInstance() {
19         return new Rectangle();
20     }
21 }
22 public class SquareCreator extends ShapeFactory {
23     @Override
24     public Shape getShapeInstance() {
25         return new Square();
26     }
27 }

1 // Step 5: Client code demonstration
2 public class FactoryMethodDemo {
3
4     public static void main(String[] args) {
5         System.out.println("==== Factory Method Design Pattern
6         =====");
7         // set the type you want
8         ShapeType shapeType = ShapeType.SQUARE;
9         // get the shape
10        Shape shape = getShapeInstance(shapeType);
11        // perform operations
12        shape.draw();
13        shape.computeArea();
14    }
15
16    private static Shape getShapeInstance(ShapeType shapeType) {
17        Shape shape = null;
18        if (shapeType == null) {
19            return null;
20        }
21        switch (shapeType) {
22            case CIRCLE:
23                ShapeFactory circleCreator = new CircleCreator();
24                shape = circleCreator.createShape();
25                break;
26            case RECTANGLE:
27                ShapeFactory rectangleCreator = new
28                RectangleCreator();
29                shape = rectangleCreator.createShape();
30                break;
31            case SQUARE:
32                ShapeFactory squareCreator = new SquareCreator();
33                shape = squareCreator.createShape();
34                break;
35            default:
36                throw new IllegalStateException("ShapeType doesn't
37                exist.");
38        }
39        return shape;
40    }
41 }

```

Factory Method vs Simple Factory

Simple Factory (shown in the example):

- Static method that creates objects based on parameters
- Not a true design pattern, more of a programming idiom
- Violates the Open/Closed principle when adding new types

Factory Method Pattern (shown in the example):

- Uses inheritance and polymorphism
- Each concrete creator handles one product type

- Follows the Open/Closed principle perfectly

 The Factory Method Pattern is especially useful in frameworks and libraries that require extension points for users to customize object creation behavior.

Concept && Coding

Object Pool

Definition

The Problem (Ex: DBConnection Pool)

Code

Solution (Ex: DBConnection Pool)

Class Diagram

Implementation

Advantages

Disadvantages

▼ Resources

- Video → [43. LLD: Object Pool Design Pattern | Creational Design Pattern | Low Level Design](#)

Definition

The **Object Pool Design Pattern** is a **creational pattern** that manages a set (pool) of reusable objects (like DBConnection Objects). Instead of creating and destroying objects frequently, we **reuse pre-created instances** from a pool.

❗ Borrow an instance from the pool → Use it and → Return it to the pool

Object Pool Design Pattern is used when:

- Object **creation is expensive** (CPU/memory).
- The **same type of object** is needed repeatedly.
- You want to **limit the number of instances** (e.g., database connections).

Concept && Coding

The Problem (Ex: DBConnection Pool)

Code

```
1 // Resource - Reusable Object
2 public class DBConnection {
3     Connection mySqlConnection;
4     public DBConnection() {
5         try {
6             mySqlConnection = DriverManager
7                 .getConnection("jdbc:mysql://localhost:3306/DB",
8 "root", "root");
9         } catch (Exception e) {
10             e.printStackTrace();
11         }
12     }
13 }
```

```
1 // Object Pool
2 public class DBConnectionPoolManager {
3     List<DBConnection> freeConnections = new ArrayList<>();
4     List<DBConnection> inUseConnections = new ArrayList<>();
5     int INITIAL_POOL_SIZE = 3;
6     int MAX_POOL_SIZE = 6;
7
8     public DBConnectionPoolManager() {
```

```

9         for (int i = 0; i < INITIAL_POOL_SIZE; i++) {
10             freeConnections.add(new DBConnection());
11         }
12     }
13
14     public DBConnection getDBConnection() {
15         DBConnection dbConnection = null;
16         if (freeConnections.isEmpty() && inUseConnections.size() <
17             MAX_POOL_SIZE) {
18             freeConnections.add(new DBConnection());
19             System.out.println("New DBConnection created and added to
20             freeConnections list.");
21             System.out.println("freeConnections size: " +
22             freeConnections.size());
23             System.out.println("inUseConnections size: " +
24             inUseConnections.size());
25             } else if (freeConnections.isEmpty() &&
26             inUseConnections.size() >= MAX_POOL_SIZE) {
27                 System.out.println("Pool is full. Cannot create new
28                 DBConnection.");
29                 return null;
30             }
31             dbConnection = freeConnections.remove(freeConnections.size() -
32             1);
33             inUseConnections.add(dbConnection);
34             System.out.println("DBConnection retrieved from
35             freeConnections list and added to inUseConnections list.");
36             System.out.println("freeConnections size: " +
37             freeConnections.size());
38             System.out.println("inUseConnections size: " +
39             inUseConnections.size());
40             return dbConnection;
41         }
42     }
43
44     public void releaseDBConnection(DBConnection dbConnection) {
45         if (dbConnection != null) {
46             inUseConnections.remove(dbConnection);
47             freeConnections.add(dbConnection);
48             System.out.println("DBConnection released from
49             inUseConnections list and added to freeConnections list.");
50             System.out.println("freeConnections size: " +
51             freeConnections.size());
52             System.out.println("inUseConnections size: " +
53             inUseConnections.size());
54             } else {
55                 System.out.println("DBConnection is null. Cannot
56                 release.");
57             }
58         }
59     }

```

```

1 // Client - Object Pool Problem Demo
2 public class Client {
3     public static void main(String[] args) {
4         // Creating a DBConnectionPoolManager
5         DBConnectionPoolManager poolManager = new
6         DBConnectionPoolManager();
7
8         // Creating 6 DBConnections (MAX_POOL_SIZE is 6)
9         DBConnection dbConnection1 = poolManager.getDBConnection();
10        DBConnection dbConnection2 = poolManager.getDBConnection();
11        DBConnection dbConnection3 = poolManager.getDBConnection();
12        DBConnection dbConnection4 = poolManager.getDBConnection();
13        DBConnection dbConnection5 = poolManager.getDBConnection();
14        DBConnection dbConnection6 = poolManager.getDBConnection();
15
16        // 7th DBConnection will not be created as the pool is full
17        (returns null)
18        DBConnection nullDBConnection = poolManager.getDBConnection();
19        System.out.println(nullDBConnection == null ? "DBConnection is
20        null as POOL is full." : "DBConnection is not null");

```

```

18     poolManager.releaseDBConnection(dbConnection6); // Releasing a
    DBConnection
19     DBConnection dbConnection = poolManager.getDBConnection(); //
    Reusing the released DBConnection
20
21     // ***** Issues with this code *****
22     // What happens if another client tries to create a new
    DBConnectionPoolManager?
23     DBConnectionPoolManager poolManager2 = new
    DBConnectionPoolManager();
24     // more connections added to the pool that exceeds the
    MAX_POOL_SIZE
25     System.out.println("===== Same Instance? =====");
26     System.out.println(poolManager == poolManager2 ? "Same
    instance of DBConnectionPoolManager" : "Different instances of " +
27         "DBConnectionPoolManager");
28
29 }
30 }

```

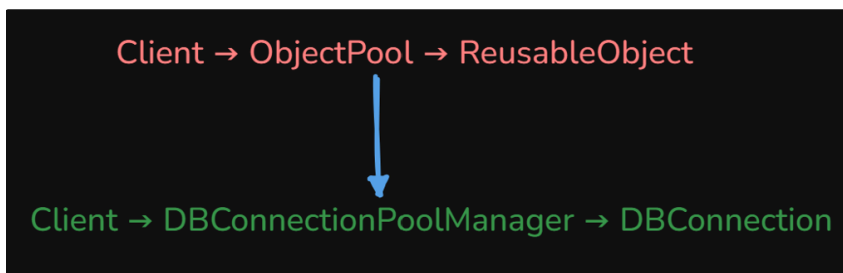
❌ What's **wrong** with the above code? What happens if another client tries to create a new `DBConnectionPoolManager` ?

- More connections were added to the pool that exceed the `MAX_POOL_SIZE` .
- Multiple connection list objects are being created to track connection usage.
- The system will eventually lead to a memory leak.
- Unreliable design.

Solution (Ex: DBConnection Pool)

✅ This **Object Pool Design Pattern** is used in conjunction with the **Singleton Design Pattern** and requires **thread safety** when acquiring and releasing resources.

Class Diagram



- **ObjectPool** (`ObjectPool`): Maintains available and in-use objects.
- **ReusableObject** (`DBConnection`): The object being pooled.

- **Client (Client)**: Uses and returns objects from the pool.

Implementation

```
1 // Resource - Reusable Object
2 public class DBConnection {
3     Connection mySqlConnection;
4
5     public DBConnection() {
6         try {
7             mySqlConnection = DriverManager
8                 .getConnection("jdbc:mysql://localhost:3306/DB",
9 "root", "root");
10        } catch (Exception e) {
11            e.printStackTrace();
12        }
13    }
14 }
15
16 // Object Pool - Singleton Implementation
17 public class DBConnectionPoolManager {
18
19     // Singleton
20     private static DBConnectionPoolManager
21     dbConnectionPoolManagerInstance = null;
22
23     List<DBConnection> freeConnections = new ArrayList<>();
24     List<DBConnection> inUseConnections = new ArrayList<>();
25     int INITIAL_POOL_SIZE = 3;
26     int MAX_POOL_SIZE = 6;
27
28     // private constructor
29     private DBConnectionPoolManager() {
30         for (int i = 0; i < INITIAL_POOL_SIZE; i++) {
31             freeConnections.add(new DBConnection());
32         }
33     }
34
35     // Singleton - thread-safe double-checked locking
36     public static DBConnectionPoolManager getInstance() {
37         if (dbConnectionPoolManagerInstance == null) {
38             synchronized (DBConnectionPoolManager.class) {
39                 if (dbConnectionPoolManagerInstance == null) {
40                     dbConnectionPoolManagerInstance = new
41 DBConnectionPoolManager();
42                 }
43             }
44         }
45         return dbConnectionPoolManagerInstance;
46     }
47
48     // Thread-safe: Only one thread can access this method at a time
49     // and modify the freeConnections and inUseConnections lists
50     public synchronized DBConnection getDBConnection() {
51         DBConnection dbConnection = null;
52         if (freeConnections.isEmpty() && inUseConnections.size() <
53 MAX_POOL_SIZE) {
54             freeConnections.add(new DBConnection());
55             System.out.println("New DBConnection created and added to
56 freeConnections list.");
57             System.out.println("freeConnections size: " +
58 freeConnections.size());
59             System.out.println("inUseConnections size: " +
60 inUseConnections.size());
61         } else if (freeConnections.isEmpty() &&
62 inUseConnections.size() >= MAX_POOL_SIZE) {
63             System.out.println("Pool is full. Cannot create new
64 DBConnection.");
65             return null;
66         }
67     }
68 }
```

```

43     }
44     dbConnection = freeConnections.remove(freeConnections.size() -
1);
45     inUseConnections.add(dbConnection);
46     System.out.println("DBConnection retrieved from
freeConnections list and added to inUseConnections list.");
47     System.out.println("freeConnections size: " +
freeConnections.size());
48     System.out.println("inUseConnections size: " +
inUseConnections.size());
49     return dbConnection;
50 }
51
52 // Thread-safe: Only one thread can access this method at a time
53 // and modify the freeConnections and inUseConnections lists
54 public synchronized void releaseDBConnection(DBConnection
dbConnection) {
55     if (dbConnection != null) {
56         inUseConnections.remove(dbConnection);
57         freeConnections.add(dbConnection);
58         System.out.println("DBConnection released from
inUseConnections list and added to freeConnections list.");
59         System.out.println("freeConnections size: " +
freeConnections.size());
60         System.out.println("inUseConnections size: " +
inUseConnections.size());
61     } else {
62         System.out.println("DBConnection is null. Cannot
release.");
63     }
64 }
65 }

```

```

1 // Client - Object Pool Solution Demo
2 public class Client {
3     public static void main(String[] args) {
4         System.out.println("===== Object Pool Design Pattern
=====");
5         // Creating a DBConnectionPoolManager
6         DBConnectionPoolManager poolManager =
DBConnectionPoolManager.getInstance();
7
8         // Creating 6 DBConnections (MAX_POOL_SIZE is 6)
9         DBConnection dbConnection1 = poolManager.getDBConnection();
10        DBConnection dbConnection2 = poolManager.getDBConnection();
11        DBConnection dbConnection3 = poolManager.getDBConnection();
12        DBConnection dbConnection4 = poolManager.getDBConnection();
13        DBConnection dbConnection5 = poolManager.getDBConnection();
14        DBConnection dbConnection6 = poolManager.getDBConnection();
15
16        // 7th DBConnection will not be created as the pool is full
17        (returns null)
18        DBConnection nullDBConnection = poolManager.getDBConnection();
19        System.out.println(nullDBConnection == null ? "DBConnection is
null as POOL is full." : "DBConnection is not null");
20        poolManager.releaseDBConnection(dbConnection6); // Releasing a
DBConnection
21        DBConnection dbConnection7 = poolManager.getDBConnection(); //
Reusing the released DBConnection
22
23        // ***** Solution Demo *****
24        // What happens if another client tries to create a new
DBConnectionPoolManager?
25        DBConnectionPoolManager poolManager2 =
DBConnectionPoolManager.getInstance();
26        System.out.println("===== Same Instance? =====");
27        System.out.println(poolManager == poolManager2 ? "Same
instance of DBConnectionPoolManager" : "Different instances of " +
"DBConnectionPoolManager");
28    }

```

Concept && Coding

Advantages

- Reduce the overhead of creating and destroying the frequently required object (generally resource-intensive objects).
- Reduce the latency, as it uses the pre-initialized object.
- Prevent resource exhaustion by managing the number of resource-intensive object creations.

Disadvantages

- Resource leakage can happen if the object is not handled properly and is not returned to the pool.
- Required more memory to manage the pool.
- Pool management required thread safety, which is additional overhead.
- Adds application complexity because of managing the pool.

Concept && Coding

Prototype

Definition

Example: Robot

The Problem: Student Example

Drawbacks

Solution: The Prototype Design Pattern

Class Diagram

Implementation

▼ Resources

- Video → [27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)

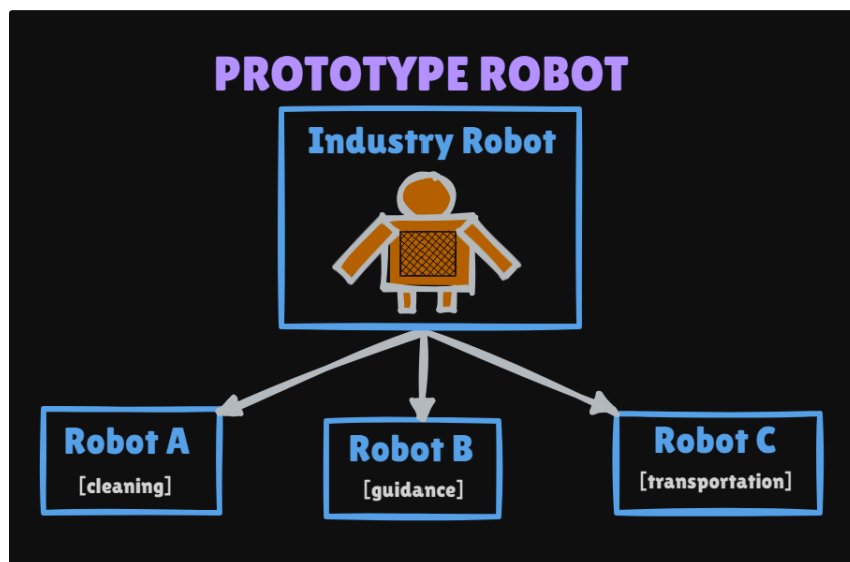
Definition

*The Prototype design pattern is a creational pattern that allows you to **create new objects by cloning existing instances** rather than creating them from scratch.*

This is particularly useful when object creation is **expensive** or when you want to avoid the complexities involved in creating objects using constructors.

Example: Robot

Concept && Coding



Creating each type of task-oriented robot from scratch is expensive. Instead, we clone the basic model of an industrial robot (the prototype) and customize it for specific tasks.

The Problem: Student Example

```
1
2 // Concrete Class - Whose clone is to be created
3 public class Student {
4
```



```

5      int id;
6      String name;
7      String branch;
8      private int rollNo;
9
10     Student() {
11
12     }
13
14     Student(int id, String name, String branch, int rollNo) {
15         this.id = id;
16         this.name = name;
17         this.branch = branch;
18         this.rollNo = rollNo;
19     }
20
21     public void printDetails() {
22         System.out.println("== Student Details ==");
23         System.out.print(this + ": ");
24         System.out.println("Id: " + id + ", Name: " + name + ",
Branch: " + branch + ", Roll No: " + rollNo);
25     }
26 }
27
28 // Client
29 public class DemoProblem {
30     public static void main(String[] args) {
31         Student studentOrg = new Student(1, "Aman", "CSE", 123);
32         studentOrg.printDetails();
33         // create a clone of the student object
34         Student studentClone = new Student();
35         studentClone.id = studentOrg.id;
36         studentClone.name = studentOrg.name;
37         studentClone.branch = studentOrg.branch;
38         // studentClone.rollNo = studentOrg.rollNo; // Compilation
error
39     }
40 }

```

Concept & Coding

Drawbacks

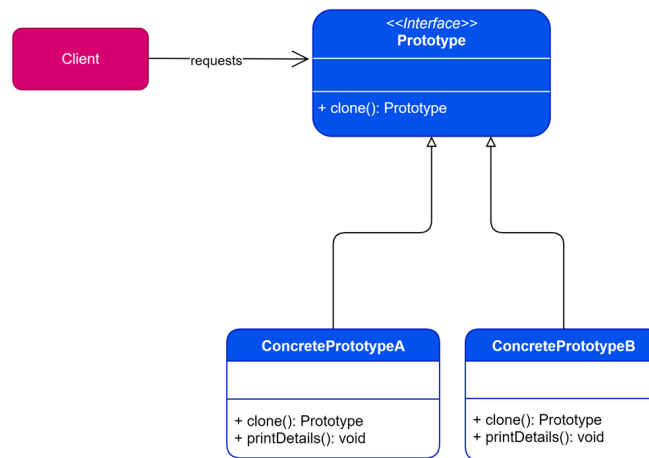
- Can't access **private** fields outside the class during cloning. Ex: **rollNo** in the **Student** class.
- It is not possible to be selective about the properties and behavior when cloning.

Solution: The Prototype Design Pattern

Instead of the client taking the responsibility of cloning the object, the cloning responsibility is assigned to the actual concrete class implementing the prototype interface to clone itself.

The prototype interface ensures consistent implementation of cloning objects with features common to all subtypes.

Class Diagram



Implementation

```
1 // Prototype interface
2 public interface StudentPrototype {
3     StudentPrototype clone();
4 }
5
6 // Concrete prototype class
7 public class Student implements StudentPrototype {
8
9     int id;
10    String name;
11    String branch;
12    boolean inHighSchool;
13    private int rollNo;
14
15    Student() {
16
17    }
18
19    // Clone constructor for efficient copying
20    Student(int id, String name, String branch, int rollNo) {
21        this.id = id;
22        this.name = name;
23        this.branch = branch;
24        this.rollNo = rollNo;
25    }
26
27    // setter method
28    public void setInHighSchool(boolean inHighSchool) {
29        this.inHighSchool = inHighSchool;
30    }
31
32    @Override
33    public StudentPrototype clone() {
34        return new Student(id, name, branch, rollNo);
35    }
36
37    public void printDetails() {
38        System.out.println("== Student Details ==");
39        System.out.print(this + ": ");
40        System.out.println("Id: " + id + ", Name: " + name + ",
Branch: " + branch + ", Roll No: " + rollNo + ", In High School: " +
inHighSchool);
41    }
42 }
43
44 // Client
45 public class DemoSolution {
46     public static void main(String[] args) {
```

Concept & Coding

```
47         // Create initial prototypes (expensive operations)
48         Student student = new Student(5, "Rita", "CSE", 224);
49         student.printDetails();
50         // Clone objects (fast operations)
51         Student studentClone = (Student) student.clone();
52         studentClone.setInHighSchool(true);
53         studentClone.printDetails();
54         System.out.println("Same object? " + (student ==
studentClone)); // false
55     }
56 }
```

Concept && Coding

Singleton

Definition

Class Diagram

Implementation of Singleton Pattern

1. Eager Initialization
2. Lazy Initialization
3. Thread-Safe/Synchronized Initialization
4. Double Locking

Issue with Double-Checked Locking

Issue 1: Instruction Reordering

Issue 2: L1 Caching

The Correct Solution: Using the volatile keyword

▼ Resources

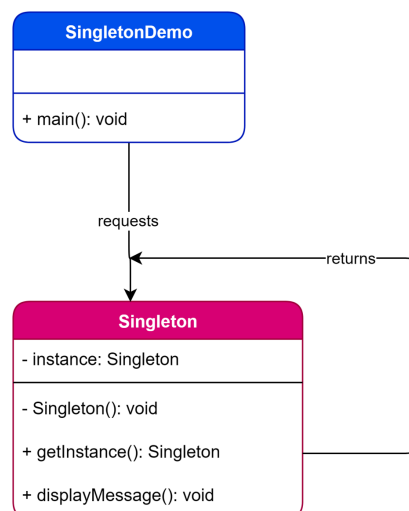
• Video → [27. All Creational Design Patterns | Prototype, Singleton, Factory, AbstractFactory, Builder Pattern](#)

• Video → [28. BUG in Double-Checked Locking of Singleton Pattern & its Fix | Low Level System Design Question](#)

Definition

The Singleton design pattern is used when we have to create only ONE instance of a class. This is useful when exactly one object is needed to perform and coordinate many actions across the system. A Singleton class ensures that there is only one instance of itself, regardless of the number of clients attempting to access it.

Class Diagram



Implementation of Singleton Pattern

There are 4 ways to implement the Singleton Pattern, each with its advantages and disadvantages.

1. Eager Initialization

- One of the simplest implementations of the Singleton Pattern.
- The instance is created when the class is loaded.
- Make the constructor **private** → prevents instantiation by other classes.
- Mark the singleton instance field as:
 - **static** variable → this ensures there's only one class instance shared across all instances of the class.
 - **final** variable → prevents the singleton instance from being reassigned after initialization.

Implementation

```
1 // 1. Eager Initialization - Singleton
2 public class DBConnectionEager {
3
4     // The single instance, created immediately
5     private static final DBConnectionEager instance = new
    DBConnectionEager();
6
7     // The private constructor prevents instantiation
8     private DBConnectionEager() {
9     }
10
11    // Method to return the unique instance of this class
12    public static DBConnectionEager getInstance() {
13        return instance;
14    }
15
16    // Method to display a message
17    public void displayMessage() {
18        System.out.println("Eager Initialization - Singleton - " +
    this);
19    }
20 }
21 // Test Singleton Implementation
22 public class SingletonDemo {
23     public static void main(String[] args) {
24         System.out.println("==== Testing Eager Initialization
    ====");
25         // SingletonObject obj = new SingletonObject(); -->
    Compilation error
26         // Get the unique instance of SingletonObject
27         DBConnectionEager eager1 = DBConnectionEager.getInstance();
28         DBConnectionEager eager2 = DBConnectionEager.getInstance();
29         // Display the message
30         eager1.displayMessage();
31         eager2.displayMessage();
32         // Check if the instances are the same
33         System.out.println("Same instance? " + (eager1 == eager2));
34     }
35 }
```

2. Lazy Initialization

- The instance is created only once when the client requests it.
- Make the constructor **private** → prevents instantiation by other classes.
- It is not thread-safe. When multiple threads access the **getInstance** method for the first time, there is a chance of multiple singleton instances being created.
- Mark the singleton instance field as:
 - **static** variable → this ensures there's only one class instance shared across all instances of the class.

Implementation

```
1 // 2. Lazy Initialization Singleton
2 public class DBConnectionLazy {
3
4     private static DBConnectionLazy instance = null;
5
6     // The private constructor prevents instantiation
7     private DBConnectionLazy() {
8     }
9
10    // Singleton Object is created only when it is required
11    // This method returns the unique instance of this class
12    // Drawback: This implementation is not thread-safe.
13    public static DBConnectionLazy getInstance() {
14        if (instance == null) {
15            instance = new DBConnectionLazy();
16        }
17        return instance;
18    }
19
20    // Method to display a message
21    public void displayMessage() {
22        System.out.println("Lazy Initialization - Singleton - " +
23        this);
24    }
25 }
26 // Test Singleton Implementation
27 public class SingletonDemo {
28     public static void main(String[] args) {
29         System.out.println("===== Testing Lazy Initialization =====");
30         // Get the unique instance of SingletonObject
31         DBConnectionLazy lazy1 = DBConnectionLazy.getInstance();
32         DBConnectionLazy lazy2 = DBConnectionLazy.getInstance();
33         // Display the message
34         lazy1.displayMessage();
35         lazy2.displayMessage();
36         // Check if the instances are the same
37         System.out.println("Same instance? " + (lazy1 == lazy2)); //
38         true
39     }
40 }
```

3. Thread-Safe/Synchronized Initialization

- This is similar to Lazy Initialization
- It is made thread-safe by using the **synchronized** keyword in the **getInstance** method, ensuring only one thread can execute this method at a time.
- This will ensure only one singleton instance is created when multiple threads are invoking the same **getInstance** method.
- But this process of singleton instantiation can be expensive because when a thread enters the synchronized method, it acquires a **lock** on the **class object**, does the job, and releases the lock for other threads to acquire it and execute the method.
 - Imagine 100 threads invoking the synchronized **getInstance** method simultaneously; 99 will have to wait until the first thread finishes its execution.

Implementation

```
1 // 3. Thread Safe Singleton
2 public class DBConnectionThreadSafe {
3
4     // Singleton Object is created only when it is required
```

```

5     private static DBConnectionThreadSafe instance = null;
6
7     // Private Constructor to avoid client applications from using the
    constructor
8     private DBConnectionThreadSafe() {
9     }
10
11    // Thread Safe Method to return the unique instance of this class
12    public static synchronized DBConnectionThreadSafe getInstance() {
13        if (instance == null) {
14            instance = new DBConnectionThreadSafe();
15        }
16        return instance;
17    }
18
19    // Method to display a message
20    public void displayMessage() {
21        System.out.println("Thread Safe Singleton - " + this);
22    }
23 }
24 // Test Singleton Implementation
25 public class SingletonDemo {
26     public static void main(String[] args) {
27         System.out.println("===== Testing Thread Safe =====");
28         // Get the unique instance of SingletonObject
29         DBConnectionThreadSafe threadSafe1 =
    DBConnectionThreadSafe.getInstance();
30         DBConnectionThreadSafe threadSafe2 =
    DBConnectionThreadSafe.getInstance();
31         // Display the message
32         threadSafe1.displayMessage();
33         threadSafe2.displayMessage();
34         // Check if the instances are the same
35         System.out.println("Same instance? " + (threadSafe1 ==
    threadSafe2)); //true
36     }
37 }

```

Concept && Coding

4. Double Locking

- It is a more optimised version of thread-safe singleton object instantiation.
- This method reduces performance overhead from synchronization(as seen previously) by only synchronizing the block of code when the singleton object is initially created(the first time). Upon instantiation, the other/future threads do not have to enter the synchronized block. Thus, making it faster.
- Widely used in the industry.

Implementation

```

1 // 4. Double Locking Singleton
2 public class DBConnectionDoubleLocking {
3
4     // Double Locking Singleton instance variable
5     private static DBConnectionDoubleLocking instance = null;
6
7     // Private constructor
8     private DBConnectionDoubleLocking() {
9     }
10
11    // Thread Safe Method to return the unique instance of this class
12    public static DBConnectionDoubleLocking getInstance() {
13        if (instance == null) { // first check
14            synchronized (DBConnectionDoubleLocking.class) {
15                if (instance == null) { // second check
16                    instance = new DBConnectionDoubleLocking();
17                }
18            }
19        }
20    }
21 }

```

```

20         return instance;
21     }
22
23     // Method to display a message
24     public void displayMessage() {
25         System.out.println("Double Locking Singleton - " + this);
26     }
27 }
28 // Test Singleton Implementation
29 public class SingletonDemo {
30     public static void main(String[] args) {
31         System.out.println("===== Testing Double Locking =====");
32         // Get the unique instance of SingletonObject
33         DBConnectionDoubleLocking doubleLocking1 =
DBConnectionDoubleLocking.getInstance();
34         DBConnectionDoubleLocking doubleLocking2 =
DBConnectionDoubleLocking.getInstance();
35         // Display the message
36         doubleLocking1.displayMessage();
37         doubleLocking2.displayMessage();
38         // Check if the instances are the same
39         System.out.println("Same instance? " + (doubleLocking1 ==
doubleLocking2)); // true
40
41     }
42 }

```

Issue with Double-Checked Locking

Video explanation → [28. BUG in Double-Checked Locking of Singleton Pattern & its Fix | Low Level System Design Quest](#)

ion

Consider the following example with a member variable `portNumber` :

Concept & Coding

```

1 // Singleton - Double Checked Locking - Issue Demo
2 public class DBConnectionDoubleCheckedLockIssue {
3
4     // Double Locking Singleton instance variable
5     private static DBConnectionDoubleCheckedLockIssue connectionObj =
null;
6     int portNumber;
7
8     // Private constructor
9     private DBConnectionDoubleCheckedLockIssue(int portNumberValue) {
10         portNumber = portNumberValue;
11     }
12
13     // Thread Safe Method to return the unique instance of this class
14     public static DBConnectionDoubleCheckedLockIssue
getConnectionObj() {
15         if (connectionObj == null) { // First check
16             synchronized (DBConnectionDoubleCheckedLockIssue.class) {
17                 if (connectionObj == null) { // Second check
18                     connectionObj = new
DBConnectionDoubleCheckedLockIssue(5567);
19                 }
20             }
21         }
22         return connectionObj;
23     }
24
25     // Method to display a message
26     public void displayMessage() {
27         System.out.println("Singleton - Double Checked Locking - Issue
- " + this);
28     }
29 }

```



```

public static DBConnectionDoubleCheckedLockIssue getConnectionObj() {
    if (connectionObj == null) { // First check
        synchronized (DBConnectionDoubleCheckedLockIssue.class) {
            if (connectionObj == null) { // Second check
                connectionObj = new DBConnectionDoubleCheckedLockIssue( portNumberValue: 5567);
            }
        }
    }
    return connectionObj;
}

```

The above highlighted step(creation of object) involves the following steps:

1. Allocating memory
2. Initializing the object with all member variables
3. Assigning the reference

Issue 1: Instruction Reordering

The JVM can reorder the object construction steps to achieve better performance. The JVM might **assign the reference before fully initializing the object**, causing other threads to see a **partially constructed object**.

```

public static DBConnectionDoubleCheckedLock getConnectionObj() {
    if (connectionObj == null) { // First check
        synchronized (DBConnectionDoubleLocking2.class) {
            if (connectionObj == null) { // Second check
                L1 memoryPointer = allocateMemory();
                L2 memoryPointer.portNumber= initializeVariable();
                L3 connectionObj = memoryPointer;
            }
        }
        return connectionObj;
    }
}

```

⚠ While T1 is executing the reordered instructions, when T1 is executing L3, let's say T2 performs the first check, it gets the **connectionObject** i.e. **NOT NULL** and the **portNumber** holds the default value(after execution of L2 by T1) and proceeds with performing operations using the partially constructed **connectionObject**, which can result in an error as T1 is yet to initialize the **portNumber** (member variable).

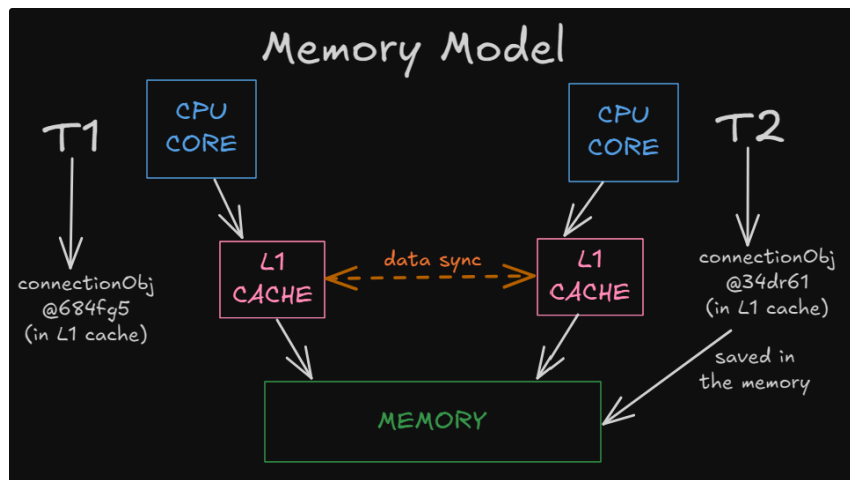
Issue 2: L1 Caching

Without proper synchronization, changes made by one thread may not be visible to other threads because of CPU caching and memory models.

```

public static DBConnectionDoubleCheckedLockIssue getConnectionObj() {
    if (connectionObj == null) { // First check
        synchronized (DBConnectionDoubleCheckedLockIssue.class) {
            if (connectionObj == null) { // Second check
                connectionObj = new DBConnectionDoubleCheckedLockIssue( portNumberValue: 5567);
            }
        }
    }
    return connectionObj;
}

```



⚠ In a multicore CPU, each CPU core has its own cache; sometimes, cores might not synchronize their caches, leading to inconsistent views of memory. When T1 creates a fully constructed `connectionObject` and saves it in the L1 cache, and if another thread T2 requests a `connectionObject` before the Caches have been synchronised and the changes have been saved in memory, T2 will proceed to create another singleton `connectionObject` instance and perform operations with it. This will lead to multiple singleton instances.

The Correct Solution: Using the `volatile` keyword

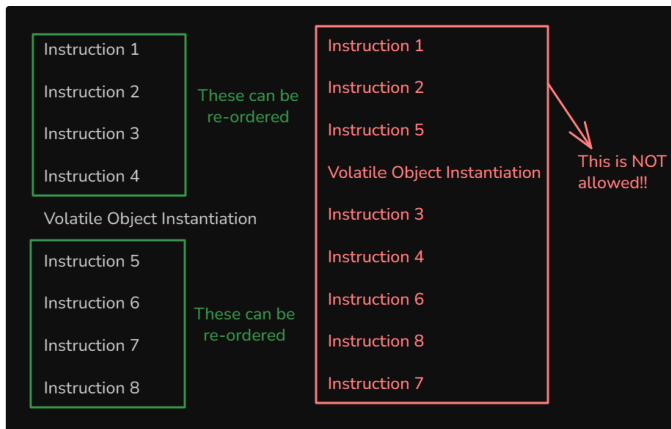
Understanding the `volatile` keyword

When a variable is declared as `volatile`, it instructs the JVM and the compiler to handle that variable in a specific way. This modifier is primarily used in multithreaded programming to ensure memory visibility, consistency of shared variables across different threads, and instruction ordering.

How do `volatile` keyword properties solve the issues with double-checked locking?

- **Memory Visibility Guarantee:** All reads and writes of a `volatile` variable are always performed directly to and from the main memory, and all write operations are immediately visible to all threads. Without `volatile`, threads might cache variables locally and not see updates from other threads. [\[Solves Issue 2\]](#)
- **Instruction Re-ordering:** The `volatile` keyword provides guarantees to prevent specific types of instruction reordering, as illustrated below. This prevention is accomplished by establishing "happens-before" guarantees, which effectively create memory barriers or fences. These barriers stop the compiler and CPU from reordering instructions around `volatile` reads or writes, even if such reordering would enhance performance in a single-threaded context.

[\[Solves Issue 1\]](#)



```
public static DBConnectionDoubleCheckedLock getConnectionObj() {
    if (connectionObj == null) { // First check
        synchronized (DBConnectionDoubleLocking2.class) {
            if (connectionObj == null) { // Second check
                memoryPointer = allocateMemory();
                memoryPointer.portNumber = initializeVariable();
                connectionObj = memoryPointer;
            }
        }
    }
    return connectionObj;
}
```

only these two instructions are allowed to be reordered (but, in this cases its not logical to re-order these two)

volatile object instruction

Implementation: Using the volatile keyword

```
1 // Singleton - Double Checked Locking - Fix Demo
2 public class DBConnectionDoubleCheckedLockFix {
3
4     // volatile keyword is used to ensure that the value of the
    variable
5     // is fetched from the memory every time.
6     private static volatile DBConnectionDoubleCheckedLockFix
    connectionObj = null;
7
8     int portNumber;
9
10    private DBConnectionDoubleCheckedLockFix(int portNumberValue) {
11        portNumber = portNumberValue;
12    }
13
14    // Thread Safe Method to return the unique instance of this class
15    public static DBConnectionDoubleCheckedLockFix
    getConnectionObj(int portNumberValue) {
16        if(connectionObj == null) {
17            synchronized(DBConnectionDoubleCheckedLockFix.class) {
18                if(connectionObj == null) {
19                    connectionObj = new
    DBConnectionDoubleCheckedLockFix(portNumberValue);
20                }
21            }
22        }
23        return connectionObj;
24    }
25
26    // Method to display a message
27    public void displayMessage() {
28        System.out.println("Singleton - Double Checked Locking - Fix -
    " + this);
29    }
30 }
```